



THE HEART CENTER · INFORMATION TECHNOLOGY

Intranet Platform

Technical Documentation & Change Management
Reference

Document Number	HCI-IT-CMD-001
Version	1.0
Effective Date	June 01, 2026
Classification	Internal Use Only
Owner	IT Operations
Review Cycle	Annual or upon major change
Production Host	Huntsville Hospital VMware vSphere (single VM)
Production URL	https://Intranet-HCI.heart.local Database: same VM, port 5432

Document Control

Authorship & Approval

Role	Name	Signature	Date
Prepared by			
Technical Reviewer			
IT Director / Approver			
Security Reviewer			
Database Administrator			

Revision History

Version	Date	Author	Description of Change
1.0	June 01, 2026	IT Operations	Initial production release. Documents the deployment of the Heart Center Intranet on a single Huntsville Hospital VMware vSphere VM, with the Fastify backend in Docker, PostgreSQL 16 running natively on the host for direct DBA access, and TLS-protected access at https://Intranet-HCI.heart.local .

Distribution List

Role	Distribution Method
IT Operations Team	Internal git repository
IT Director	Email / shared drive
Database Administrators	Hospital shared drive
Designated Backup Operator	Printed copy in operations binder

Related Documents

Reference	Title
HCI-IT-CMD-001-A	Huntsville Hospital Information Security Policy
HCI-IT-CMD-001-B	Incident Response Plan
HCI-IT-CMD-001-C	Acceptable Use Policy
HCI-IT-CMD-001-D	Change Management Procedure
HCI-IT-CMD-001-E	PostgreSQL Administration Standard

Purpose and Scope

Purpose. This document provides the canonical technical reference for The Heart Center employee intranet platform as deployed in the Huntsville Hospital production environment. It defines the production architecture, documents operational procedures, codifies recovery actions, and serves as the change-management baseline for the system.

Scope. Covers the Red Hat Enterprise Linux 10 virtual machine that hosts the intranet application on the Huntsville Hospital VMware vSphere cluster, the containerised application stack (nginx + Node.js / Fastify + Prisma + React), TLS-protected remote-access mechanisms, and the maintenance and recovery procedures. Does not cover the Huntsville Hospital network infrastructure, the vSphere cluster management, or PostgreSQL server-level administration beyond the scope of this single database.

Intended audience. IT Operations personnel, the IT Director, the Security Reviewer, Database Administrators, and designated backup operators.

Change control. Any modification to the production system that invalidates a procedure in this document constitutes a change requiring an updated revision of this document. Update the Revision History table when this occurs. All schema changes flow through Prisma migrations committed to source control; ad-hoc schema modifications on the database VM are prohibited.

1 01 — System Overview

1.1 What this system is

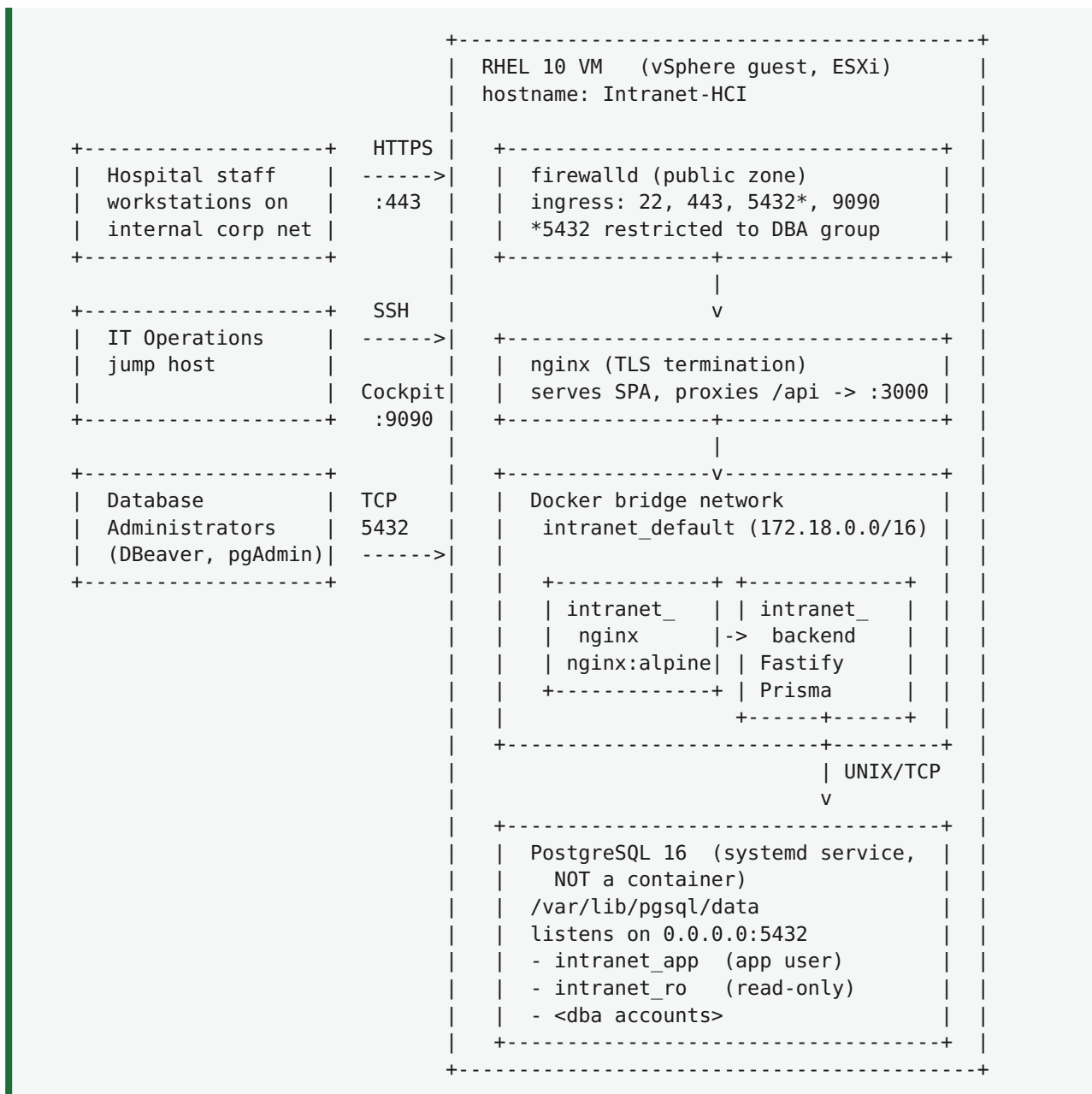
The **Heart Center Intranet** is an internal-only web platform for The Heart Center medical practice. It provides employees with:

- Company announcements and news posts (with administrative moderation)
- Employee directory and search
- HR resources, policies, IT support requests
- Shared schedules, sign-up sheets, and event coordination
- Recognition programs (Employee Spotlight nominations)
- Centralized links to web applications used by the practice

In production it is deployed as a single Red Hat Enterprise Linux 10 virtual machine running on **Huntsville Hospital's VMware vSphere** infrastructure, reached over the hospital's internal corporate network at the HTTPS endpoint `https://Intranet-HCI.heart.local`.

1.2 High-level architecture

The entire platform runs on a single VM. The Fastify application runs inside Docker; PostgreSQL runs natively on the host so DBAs can reach it directly without any container access.



1.3 Service layers

The production system is organized in **five concentric layers**:

```

+-----+
|          (5)  USER EXPERIENCE          |
| React 19 SPA * Admin HTML portal * Manager HTML portal |
+-----+
|          (4)  APPLICATION                |
| Fastify backend (Node.js 20) * @fastify/session * Prisma |
| REST API * 4 role tiers * audit logging |
+-----+
|          (3)  DATA                     |
| PostgreSQL 16 (native systemd service, queryable externally) |
| Session store inside same Postgres database |
| Uploaded files on bind-mounted host filesystem (/uploads) |
+-----+
|          (2)  PLATFORM                  |
| nginx reverse proxy (TLS) * Docker * bind-mount volumes |
+-----+
|          (1)  INFRASTRUCTURE            |
| RHEL 10 * VMware vSphere * firewalld * Cockpit |
+-----+

```

1.4 Why the database runs on the host instead of in a container

Direct external access for DBAs. PostgreSQL listening on the host's network interface means database administrators connect from their normal workstation tools (psql, DBeaver, pgAdmin) without any `docker exec` or container-specific knowledge.

Independent lifecycle. The database starts at boot via systemd before Docker comes up. Container rebuilds, image updates, or `docker compose down` operations do not touch the database.

Standard hospital backup integration. Backup agents and monitoring that the hospital already operates against PostgreSQL clusters work unchanged against this instance.

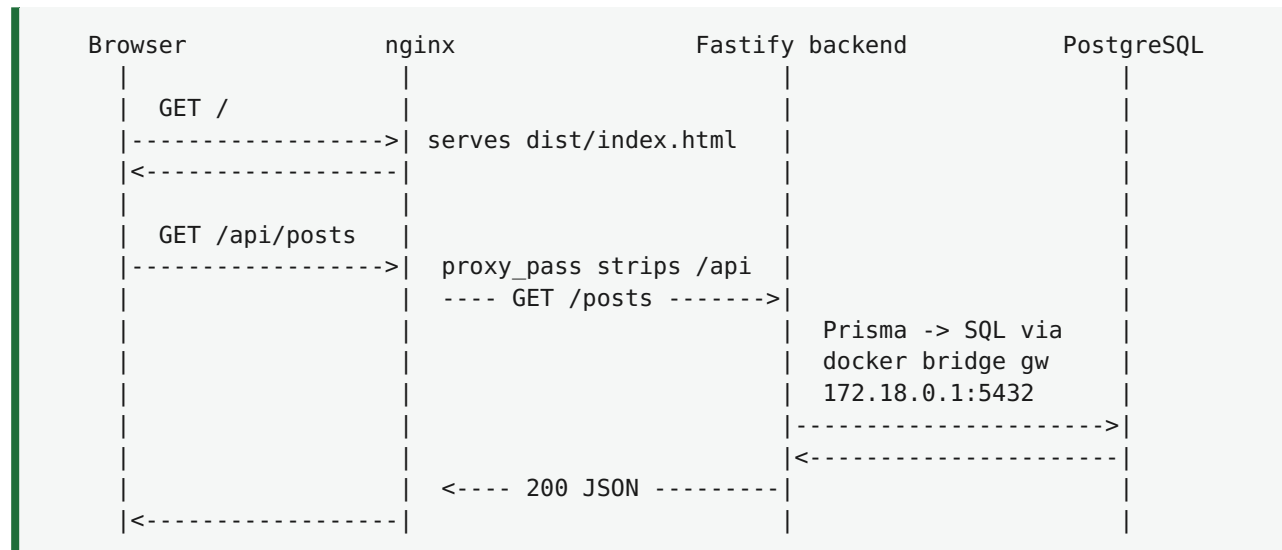
Simpler disaster recovery. Restoring PostgreSQL from a `pg_dump` or file-system snapshot does not require any container orchestration.

1.5 Audiences served

Audience	How they reach the system	Where they land
Hospital staff (any device, internal network)	Browser to <code>https://Intranet-HCI.heart.local/</code>	React SPA (employee view)
Managers	Browser to <code>https://Intranet-HCI.heart.local/manager/</code>	Static manager portal
Admins / Super-admins	Browser to <code>https://Intranet-HCI.heart.local/admin/</code>	Static admin portal
Technical operators	SSH from the IT-Ops jump host	Linux shell
System maintenance	HTTPS to <code>https://Intranet-HCI.heart.local:9090/</code>	Cockpit web console
Database administrators	psql / DBeaver / pgAdmin to <code>Intranet-HCI.heart.local:5432</code>	Direct SQL access

1.6 Data-flow at a glance

A typical “employee views the home page” request:



1.7 Where this lives in production

```
Huntsville Hospital vSphere Cluster
|
+- ESXi host(s)
   |
   +- VM: Intranet-HCI (single production VM)
      +- vCPU: 4
      +- vRAM: 8 GiB
      +- vDisk: 80 GiB (OS + uploads + database)
      +- vNIC: corporate VLAN
      +- DNS: Intranet-HCI.heart.local
      +- Services:
         - Docker (nginx + Fastify containers)
         - PostgreSQL 16 (systemd, on host)
         - sshd, cockpit, chronyd, firewalld
```

Continue to [02-infrastructure.md](#) for the server-side details, or jump to [06-maintenance.md](#) for operations.

2 02 — Infrastructure

This document describes the **server, operating system, network, and supporting services** that host the intranet in the Huntsville Hospital production environment.

2.1 Host inventory

Property	Value
Operating system	Red Hat Enterprise Linux 10.1 (Coughlan)
Kernel	6.12.x-el10 (current at install)
Virtualization	VMware vSphere / ESXi guest
vSphere cluster	Huntsville Hospital production cluster
VM name	Intranet-HCI
FQDN	Intranet-HCI.heart.local
Time zone	America/Chicago (host and containers)
vCPU	4
vRAM	8 GiB
vDisk	80 GiB single LVM volume /dev/mapper/rhel-root
Subscription	Red Hat Enterprise Linux subscription, registered to hospital account
Joined to	Huntsville Hospital corporate DNS; static A record managed by hospital IT

To check current numbers at any time:

```
$ hostnamectl          # OS and machine identity
$ free -h              # memory
$ df -h /              # disk
$ uptime               # how long up, load average
```

2.2 Disk layout

```

/dev/mapper/rhel-root <-- single LVM logical volume
|
+- mounted at /
  |
  +- /home/<ops-user>/          (admin user's home directory)
  |
  +- /srv/intranet/           (APPLICATION - this project)
  |   +- app/                  (Node backend source + node_modules)
  |   +- frontend/            (React source + built dist/)
  |   +- nginx/               (reverse-proxy config + TLS certs)
  |   +- public/              (admin/ and manager/ static portals)
  |   +- uploads/             (uploaded files - live)
  |   +- prisma/              (schema.prisma, migrations)
  |   +- docs/                (this documentation)
  |
  +- /var/lib/docker/         (Docker images and overlay storage)
  +- /var/lib/pgsql/data/     (PostgreSQL 16 database files)
  +- /var/log/                (System logs - journald)
  +- /etc/                    (System configuration)

```

PostgreSQL stores its data in `/var/lib/pgsql/data` on the host. The application code, uploaded files, and the database all live on this single VM but the database runs as a native systemd service, not inside a Docker container, so DBAs can connect to it directly from their workstations.

2.3 Network topology

The VM has **two logical network interfaces** in production:

```

+-----+
|               RHEL 10 VM               |
| lo          127.0.0.1/8      <-- localhost only |
| ens192      <vsphere-assigned-ip>/24          |
|              Huntsville Hospital corporate VLAN |
|              DNS: Intranet-HCI.heart.local      |
| docker0     172.18.0.1/16    <-- Docker bridge  |
|              (container-to-container only)      |
+-----+

```

Interface	Purpose	Reachable from
lo	Loopback	the VM itself
ens192	Primary NIC on the corporate VLAN	Hospital workstations, IT-Ops jump host, DBA workstations
docker0	Inter-container traffic	Containers only

There is **no Tailscale or other overlay network in production**. All access is over the hospital's internal corporate network only. The system is not exposed to the public internet.

2.4 Firewall (firewalld)

firewalld is active using the default public zone, bound to ens192. Open ingress ports in production:

Port	Protocol	Service	Allowed from
22/tcp	TCP	OpenSSH	IT-Ops jump host (source-restricted)
443/tcp	TCP	nginx (HTTPS)	Entire corporate VLAN
9090/tcp	TCP	Cockpit web console	IT-Ops jump host (source-restricted)
5432/tcp	TCP	PostgreSQL	DBA group on corporate VLAN (source-restricted)
icmp	—	ping / discovery	Corporate VLAN

Note: port 80/tcp is **not** open. nginx redirects clients to :443 internally via a same-host listener that responds only on the loopback, not via firewalld passthrough.

To inspect or modify:

```
$ sudo firewall-cmd --list-all          # show current
$ sudo firewall-cmd --permanent --add-port=N/tcp  # open a port persistently
$ sudo firewall-cmd --reload            # apply changes
```

2.5 TLS / certificates

The site is served over HTTPS using a certificate issued by the hospital's internal Certificate Authority. Trust is pre-installed on hospital workstations, so end users see a normal padlock with no warnings.

Item	Path / value
Certificate	<code>/etc/nginx/tls/intranet-hci.crt</code>
Private key	<code>/etc/nginx/tls/intranet-hci.key</code> (mode 0600, owner root)
Issued by	Huntsville Hospital Internal CA
Renewal	Annual; coordinate with hospital PKI team 30 days before expiry
Renewal procedure	See <code>06-maintenance.md</code> -> <i>Certificate rotation</i>

2.6 Services running on the host

```

systemd --- postgresql.service    <-- PostgreSQL 16 database
      +-+ docker.service          <-- runs the intranet containers
      +-+ firewalld.service        <-- packet filtering
      +-+ sshd.service             <-- SSH access (jump host only)
      +-+ cockpit.socket           <-- on-demand Cockpit (port 9090)
      +-+ chronyd.service          <-- NTP sync to hospital NTP
      +-+ rsyslog.service           <-- ships logs to hospital SIEM
      +-+ rhsm.service / rhsmcertd <-- Red Hat subscription

```

To list all running services:

```
$ systemctl list-units --type=service --state=running
```

2.7 Listening sockets (production state)

```

PORT    BIND ADDRESS    SERVICE
22      0.0.0.0 + [::]  sshd (firewall-restricted to jump host)
443     0.0.0.0 + [::]  nginx (HTTPS, served by container)
5432    0.0.0.0 + [::]  postgresql (firewall-restricted to DBA group)
9090    *               cockpit (firewall-restricted)

```

To check anytime:

```
$ sudo ss -tlnp
```

2.8 Boot-time service order

When the VM powers on, this is roughly the sequence:

```

BIOS  -> GRUB  -> kernel  -> systemd  ---> network.target
                                           +--> firewallld
                                           +--> chronyd
                                           +--> sshd
                                           +--> rsyslog
                                           +--> postgresql (database ready
                                           |               before docker)
                                           +--> docker  (starts containers
                                           |               via Compose unit-style
                                           |               restart policy)
                                           +--> cockpit.socket

```

Containers auto-start because they have `restart: unless-stopped` in `docker-compose.yml`, and the Docker daemon starts at boot.

2.9 Cockpit (web admin)

Cockpit is a Red Hat web console that lets you do most maintenance through a browser instead of SSH. Access it at:

```
https://Intranet-HCI.heart.local:9090/
```

Cockpit uses the same TLS certificate as the main site. Log in with a hospital-domain-joined account that is a member of the `wheel` (sudo) group on the VM. From Cockpit you can:

- Watch CPU / memory / disk / network graphs
- Read system logs (journald with search and filter)
- Update software packages with a UI
- Restart services
- Open a terminal in the browser

Cockpit is the **easiest path** for someone unfamiliar with the command line.

2.10 Updates and subscription

The VM is registered with Red Hat. To check status and apply updates:

```

$ sudo subscription-manager status      # confirm registration
$ sudo dnf check-update                 # list available updates
$ sudo dnf upgrade                      # apply (asks y/n)
$ sudo dnf upgrade -y                  # apply without prompt

```

After installing updates, **check if a reboot is needed**:

```
$ sudo dnf needs-restarting -r recommended # exit 1 means reboot
```

If a reboot is needed (kernel, glibc, systemd updates), schedule it through the hospital change-management process. The Cockpit “Software Updates” page provides a friendlier UI for the same operations and flags reboot requirements.

2.10.1 Recommended update cadence

Cadence	Action
Weekly	<code>sudo dnf check-update</code> - read the list, decide what to apply
Monthly	<code>sudo dnf upgrade --security</code> - security-only updates
Quarterly	Full <code>sudo dnf upgrade</code> + reboot, during a maintenance window approved through change management

Never run `dnf upgrade` mid-workday. Some packages restart services mid-update; the intranet may briefly stop responding.

2.11 Where to go next

- `03-application.md` - the intranet code itself
- `04-deployment.md` - Docker Compose and the external database
- `05-remote-access.md` - SSH, Cockpit, DBA workstation access

3 03 — Application Architecture

This document describes **what the intranet software actually is**, how its pieces talk to one another, and how data flows through it.

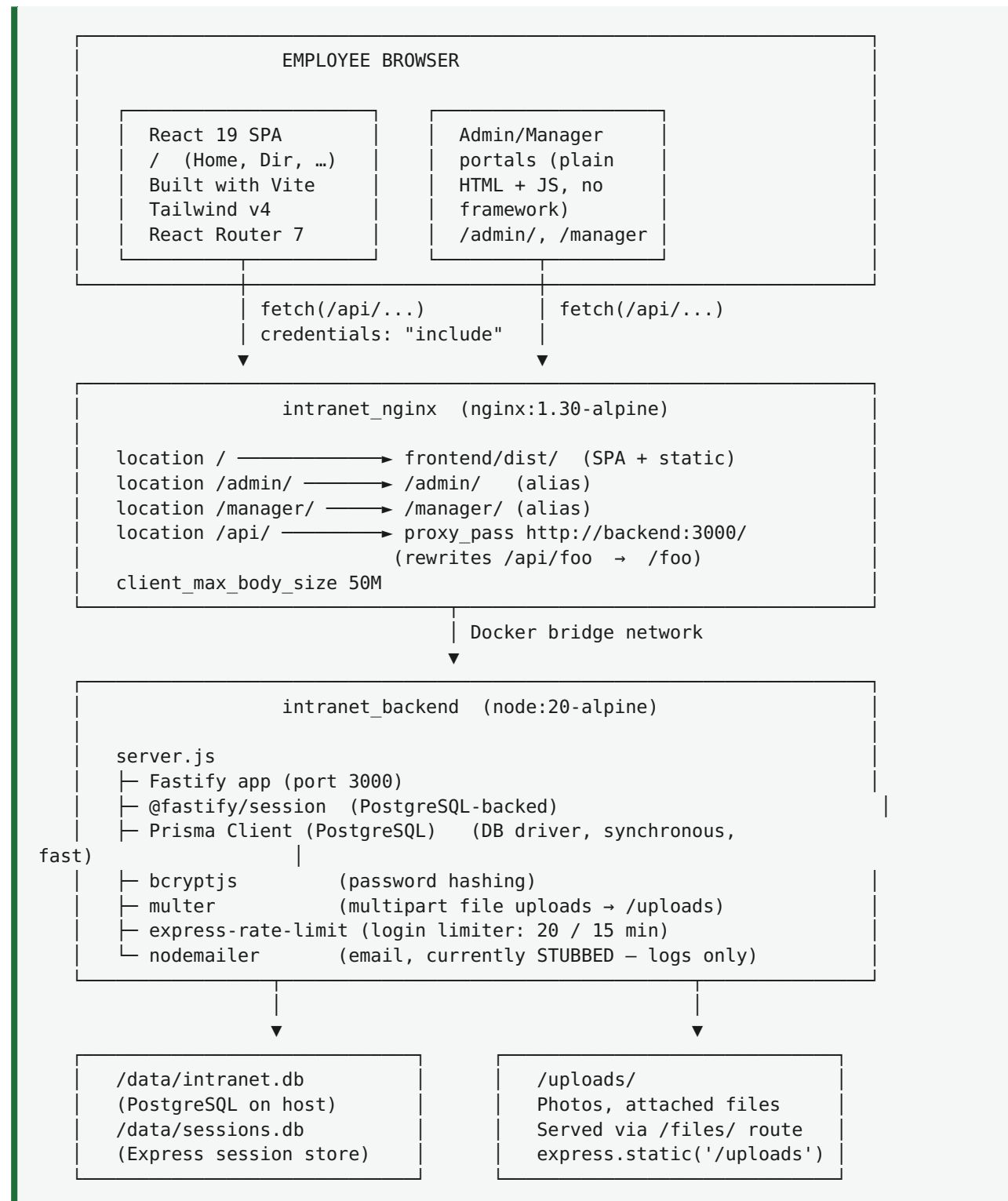
3.1 Codebase layout

```

/srv/intranet/
├── docker-compose.yml      (the two-service definition)
├── .env                    (SESSION_SECRET – sensitive)
├── app/                    ◀ BACKEND
│   ├── server.js           (Fastify entrypoint + plugin registration)
│   ├── package.json        (dependencies)
│   └── node_modules/       (installed on container boot)
├── frontend/              ◀ EMPLOYEE-FACING SPA
│   ├── src/                (React 19 source)
│   │   ├── main.jsx        (routes)
│   │   ├── App.jsx         (MainLayout)
│   │   ├── layouts/        (Home, Announcements, Directory, ...)
│   │   ├── pages/          (AnnouncementCard, EmployeeSpotlight)
│   │   ├── components/     (SettingsContext)
│   │   ├── context/        (centralized fetch wrapper)
│   │   ├── services/api.js
│   │   └── hooks/, utils/
│   ├── dist/               ◀ PRODUCTION BUILD (served by nginx)
│   ├── vite.config.js
│   └── package.json
├── public/                ◀ STATIC PORTALS (no build step)
│   ├── admin/
│   │   ├── login.html
│   │   └── dashboard.html
│   └── manager/
│       └── index.html
├── nginx/
│   └── default.conf         (reverse-proxy rules)
├── data/                  ◀ PERSISTENT DATA
│   (NOTE: in production, PostgreSQL data lives at /var/lib/pgsql/data
│   on the host – NOT inside the project tree)
├── uploads/               ◀ UPLOADED FILES (photos, resources)
└── docs/                  (you are here)

```

3.2 Component map



⚠ Because nginx **strips the** `/api` **prefix** before proxying, every route defined in `server.js` is written **without** the `/api/` prefix. For example, `app.get('/posts')` is what the browser reaches at `/api/posts`. Forgetting this is one of the most common mistakes when adding new endpoints.

3.3 Database schema

The schema is defined in `prisma/schema.prisma`. Migrations under `prisma/migrations/` apply automatically on backend startup via `prisma migrate deploy`. Prisma migrations under `prisma/migrations/` apply automatically on startup.

```
intranet_hci (PostgreSQL)
├─ users (login accounts, roles)
│   id, username, password_hash, role, created_at
├─ posts (announcements)
│   id, title, content, photo_filename, author_id,
│   status [pending|approved|rejected], reviewed_by, created_at
├─ resources (HR / Policy / etc. file listings)
├─ schedules (department schedules)
├─ spotlight (Employee Spotlight history)
├─ spotlight_nominations (incoming nominations awaiting review)
├─ directory (employee directory entries)
├─ audit_log (super-admin actions for traceability)
├─ site_settings (site-wide configuration: key/value)
├─ signup_sheets (events open for signup)
├─ signup_entries (one row per person signed up)
├─ it_tickets (IT support requests)
└─ Session ◀ table managed by @fastify/session
    (active Fastify sessions, opaque to app code)
```

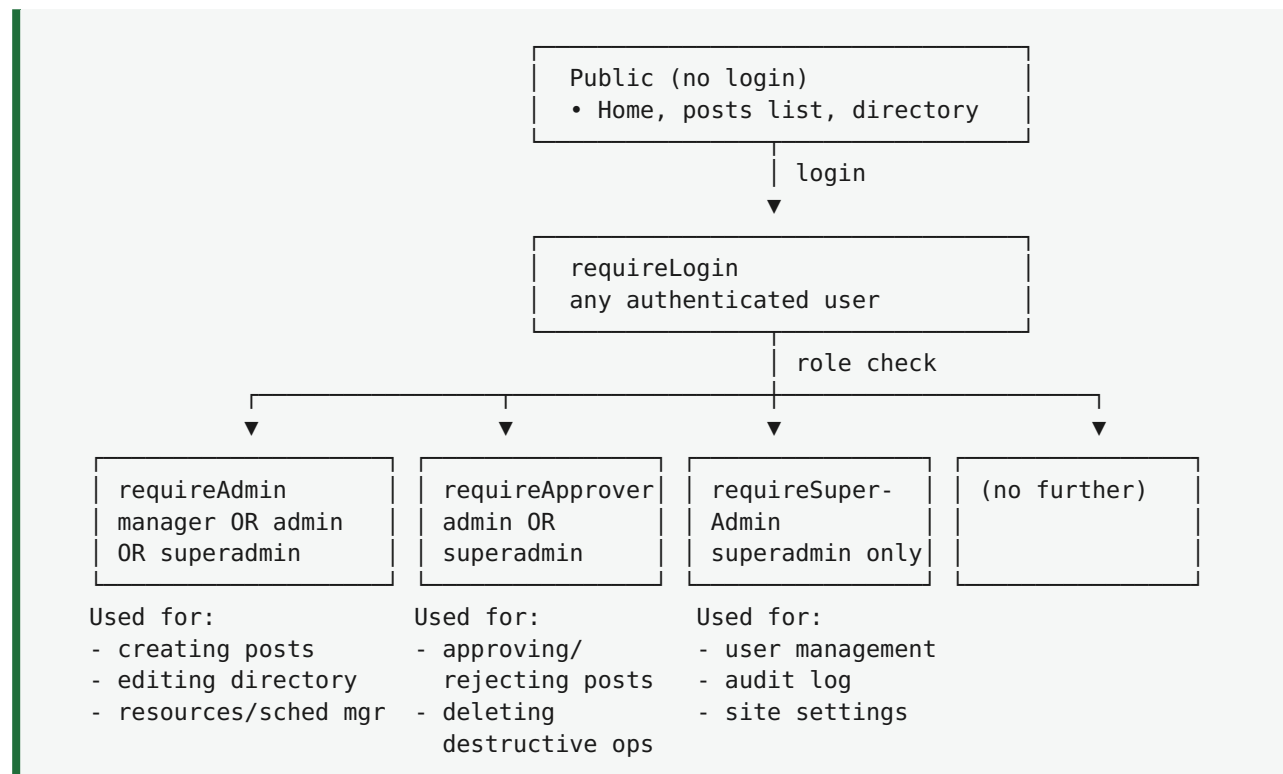
To inspect any table:

```
# From a DBA workstation (preferred):
$ psql -h Intranet-HCI.heart.local -U <dba-user> -d intranet_hci
intranet_hci=> \dt
intranet_hci=> \d "User"
intranet_hci=> SELECT id, username, role FROM "User";
intranet_hci=> \q

# Or directly on the VM as the postgres superuser:
$ sudo -u postgres psql intranet_hci
```

3.4 Roles and authorization

The application has **four authorization tiers**, implemented as middleware in `server.js`:



! Naming caveat: `requireAdmin` also lets managers in. If you intend “no managers allowed,” use `requireApprover`. This trips up new developers — the name does not match what the function does.

3.5 Frontend (React SPA) — request lifecycle

1. Browser visits `https://Intranet-HCI.heart.local/`
2. nginx returns `frontend/dist/index.html`
3. The HTML pulls `/assets/index-xxxx.js` (the bundled React app)
4. React renders `<App />`
5. `<SettingsContext>` fires `fetch('/api/settings')` on mount
 - nginx proxies to backend → returns site-wide config (announcements enabled? spotlight enabled? etc.)
6. React Router resolves the URL to a `<Page>` component
7. Page mounts → calls one of the helpers in `src/services/api.js`, e.g. `getPosts()` → `fetch('/api/posts', { credentials: 'include' })`
8. Backend reads session cookie, returns approved posts only
9. React renders the data; user sees the page

All API calls go through `src/services/api.js`. It centralizes:

- The `/api` base path
- `credentials: 'include'` so the session cookie rides along
- Error normalization (throws `Error(message)` for non-2xx)

When adding a new API endpoint, put a wrapper here rather than calling `fetch` from a component.

3.6 Admin / manager portals — why they exist

These are intentionally **not** part of the React SPA:

- They are plain HTML + vanilla JS — no build tool, no `npm install`
- Edits are immediately live; just refresh
- They have zero dependencies — survive npm-ecosystem rot

Both portals call the same `/api/*` endpoints as the SPA. They are mounted read-only into nginx at `/admin/` and `/manager/`.

Portal	Path on disk	URL
Admin	<code>/srv/intranet/public/admin/</code>	<code>http://.../admin/</code>
Manager	<code>/srv/intranet/public/manager/</code>	<code>http://.../manager/</code>

3.7 REST API surface (high-level)

A non-exhaustive list — see `server.js` for complete signatures.

```

PUBLIC
GET    /api/health                liveness
POST   /api/auth/login            { username, password }
POST   /api/auth/logout
GET    /api/auth/me               current user info
GET    /api/posts                 approved posts only
GET    /api/directory             employee directory
GET    /api/resources, /schedules, /settings, /spotlight, /search
POST   /api/it-request            open IT ticket
POST   /api/spotlight/nominations submit nomination
GET    /api/signup-sheets[:id]    list/view sign-up sheets

REQUIRE LOGIN (manager / admin / superadmin)
POST   /api/posts                 create new (status=pending)
POST   /api/resources            upload resource
POST   /api/schedules            create schedule
GET    /api/manager/my-posts      my own pending/approved
GET/POST/PATCH/DEL /api/manager/directory/...

REQUIRE APPROVER (admin / superadmin)
GET    /api/admin/posts/pending   moderation queue
POST   /api/admin/posts/:id/approve
POST   /api/admin/posts/:id/reject
DELETE /api/admin/posts/:id
GET    /api/admin/spotlight/nominations
DELETE /api/admin/spotlight/nominations/:id
DELETE /api/signup-sheets/:id

REQUIRE SUPERADMIN
GET    /api/admin/stats           dashboard counters
GET    /api/admin/audit           audit log entries
GET    /api/admin/users           user management
POST   /api/admin/users           create user
PATCH /api/admin/users/:id       update role/password
DELETE /api/admin/users/:id
POST   /api/admin/settings        site-wide settings

```

3.8 Rate limiting and security

- **Login** endpoint is rate-limited to **20 attempts per 15 minutes** per IP.
- Passwords are hashed with **bcrypt** before storage.
- Sessions are HTTP-only cookies (`httpOnly: true` , `sameSite: 'lax'`). `secure: false` is set because nginx fronts the app over plain HTTP — see the security note below.
- nginx is configured with `trust proxy = 1` so the backend honors `X-Real-IP` for accurate per-IP rate limiting.

⚠ Security gap to be aware of: the site is currently served over plain HTTP on port 8080. Credentials traverse the local network in cleartext. Acceptable on a trusted LAN, but **adding TLS is a recommended next step** (see `06-maintenance.md` → *Hardening checklist*).

3.9 Email

`nodemailer` is configured with `jsonTransport` — emails are logged to stdout instead of being delivered. The intended Exchange SMTP block is **commented out at the top of `server.js`**. To enable real email:

1. Edit `/srv/intranet/app/server.js`
2. Replace the `jsonTransport` line with one of the templates in the comment block
3. Fill in `host`, `port`, `auth`
4. `cd /srv/intranet && docker compose restart backend`

3.10 State that lives outside the container

Anything that should survive container restarts is bind-mounted from the host:

Host path	Container path	Persists	Why
<code>/srv/intranet/app/</code>	<code>/app/</code>	yes	source code (live-mounted for easy edits)
(none — DB is on host)	n/a	yes	PostgreSQL data at <code>/var/lib/pgsql/data</code> (not mounted into any container)
<code>/srv/intranet/uploads/</code>	<code>/uploads/</code>	yes	uploaded files
<code>/srv/intranet/frontend/dist/</code>	<code>/usr/share/nginx/html/</code>	yes (R/O)	built SPA
<code>/srv/intranet/public/admin/</code>	<code>/admin/</code>	yes (R/O)	static admin portal
<code>/srv/intranet/public/manager/</code>	<code>/manager/</code>	yes (R/O)	static manager portal
<code>/srv/intranet/nginx/default.conf</code>	<code>/etc/nginx/conf.d/default.conf</code>	yes (R/O)	proxy config

This means: **if the containers are destroyed, no data is lost** — only the runtime is rebuilt. The host filesystem is the source of truth.

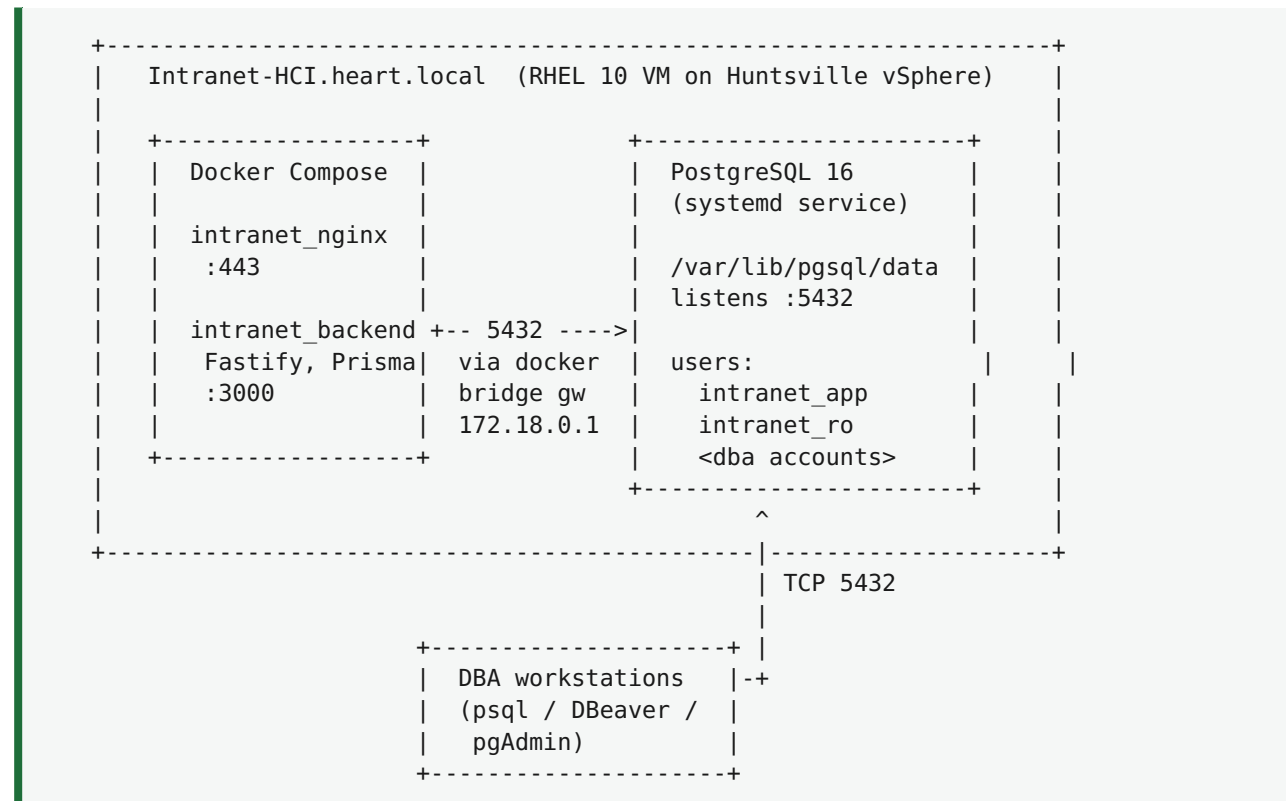
3.11 Where to go next

- `04-deployment.md` — how the containers are orchestrated
- `06-maintenance.md` — backup, rotate, and monitor the application

4 04 — Deployment

This document covers **how the intranet is packaged and run** in production — Docker Compose for the application containers, PostgreSQL as a native systemd service on the same VM, environment variables, and what happens when the system boots.

4.1 Single-VM topology



Key point: PostgreSQL is **not** in a container. It runs as a regular systemd service on the host. The Fastify backend container reaches it via the Docker bridge gateway IP (`172.18.0.1`) or, equivalently, via `host.docker.internal` when Docker is configured for it.

4.2 The Compose file

```
/srv/intranet/docker-compose.yml :
```

```

services:
  web:
    image: nginx:1.30-alpine
    container_name: intranet_nginx
    ports:
      - "443:443"
    volumes:
      - ./frontend/dist:/usr/share/nginx/html:ro
      - ./public/admin:/admin:ro
      - ./public/manager:/manager:ro
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/tls:/etc/nginx/tls:ro
    depends_on:
      - backend
    restart: unless-stopped

  backend:
    image: node:20-alpine
    container_name: intranet_backend
    working_dir: /app
    extra_hosts:
      # Lets the container reach the host as 'host.docker.internal'
      - "host.docker.internal:host-gateway"
    environment:
      - TZ=America/Chicago
      - NODE_ENV=production
      - SESSION_SECRET=${SESSION_SECRET}
      - DATABASE_URL=${DATABASE_URL}
      - INITIAL_ADMIN_PASSWORD=${INITIAL_ADMIN_PASSWORD}
    volumes:
      - ./app:/app
      - ./uploads:/uploads
    command: sh -c "npm install && npx prisma migrate deploy && npm start"
    restart: unless-stopped

```

4.2.1 Key design choices

1. **No custom Dockerfiles.** Stock public images (`nginx:1.30-alpine` and `node:20-alpine`). Application code is bind-mounted.
2. **`npm install` and `prisma migrate deploy` run on every container start.** Dependencies and schema stay in sync with the committed code at every restart.
3. **The database runs on the host, not in a container.** This is the deliberate production choice so DBAs can connect directly without container access. The backend reaches PostgreSQL through the Docker bridge.
4. **All persistent application state lives off-container.** Application code, uploads, and TLS certs are bind-mounted from the host filesystem. The database lives on the host filesystem at `/var/lib/pgsql/data` .

5. **restart: unless-stopped** on both containers — they survive Docker daemon restarts and boot automatically.
6. **extra_hosts** adds **host.docker.internal** so the connection string can read **host.docker.internal** instead of hard-coding **172.18.0.1**. Either form is acceptable.

4.3 PostgreSQL setup (one-time)

The database is provisioned once during initial deployment of the VM:

```
# 1. Install PostgreSQL 16 from the Red Hat repos
$ sudo dnf install -y postgresql-server postgresql-contrib

# 2. Initialize the data directory
$ sudo /usr/bin/postgresql-setup --initdb

# 3. Enable at boot and start now
$ sudo systemctl enable --now postgresql

# 4. Configure listen_addresses
$ sudo sed -i "s/^#listen_addresses.*/listen_addresses = '*'/" /var/lib/pgsql/data/postgresql.conf

# 5. Configure host-based authentication (pg_hba.conf)
$ sudo tee -a /var/lib/pgsql/data/pg_hba.conf <<'HBA'
# Application backend (from Docker bridge)
host    intranet_hci    intranet_app    172.18.0.0/16    scram-sha-256
host    intranet_hci    intranet_ro     172.18.0.0/16    scram-sha-256
# DBAs on the corporate VLAN
host    intranet_hci    +dba_group      <corporate-vlan>/16    scram-sha-256
HBA

# 6. Reload PostgreSQL to pick up the new config
$ sudo systemctl reload postgresql

# 7. Create database, users, and grants
$ sudo -u postgres psql <<'SQL'
CREATE USER intranet_app WITH PASSWORD '<generated-strong-password>';
CREATE USER intranet_ro  WITH PASSWORD '<generated-strong-password>';
CREATE ROLE dba_group;
CREATE DATABASE intranet_hci OWNER intranet_app;
GRANT CONNECT ON DATABASE intranet_hci TO intranet_ro;
GRANT CONNECT ON DATABASE intranet_hci TO dba_group;
SQL

# 8. Open the firewall for DBA access (restricted to the corporate VLAN)
$ sudo firewall-cmd --permanent --add-port=5432/tcp
$ sudo firewall-cmd --reload
```

After step 7 the Prisma schema is applied automatically by the backend on its first start (`npx prisma migrate deploy`).

4.4 Container lifecycle

```

$ docker compose up -d
      |
      v
+-----+
| Pull images (if not cached) |
+-----+
      |
      v
+-----+
| Create network 'intranet_default' (172.18.0.0/16) |
+-----+
      |
      v
+-----+
| Start backend container |
|   - mount ./app, ./uploads |
|   - run npm install |
|   - run npx prisma migrate deploy |
|       (connects to PostgreSQL on the host) |
|   - run npm start -> node server.js |
|   - server binds 0.0.0.0:3000 inside container |
+-----+
      |
      v
+-----+
| Start web container (TLS termination, port 443) |
+-----+
      |
      v
READY

```

4.5 Environment & secrets

Secrets live in `/srv/intranet/.env` on the VM:

```

SESSION_SECRET=<48-byte hex string>
DATABASE_URL=postgresql://intranet_app:<password>@host.docker.internal:5432/
intranet_hci?schema=public&sslmode=disable
INITIAL_ADMIN_PASSWORD=<set once for first-boot seeding, then rotated via UI>

```

Note that `sslmode=disable` is acceptable here because the connection never leaves the VM — the Docker bridge interface is internal to the host kernel. For DBA workstations connecting from elsewhere on the network, configure client-side TLS or `sslmode=require` depending on hospital policy.

The backend **refuses to start** if `SESSION_SECRET` or `DATABASE_URL` is empty.

To rotate the session secret (invalidates all active sessions):

```
$ openssl rand -hex 48 # generate new value
$ sudo nano /srv/intranet/.env # paste in
$ docker compose restart backend # apply
```

To rotate the application user's database password:

```
$ sudo -u postgres psql -c "ALTER USER intranet_app WITH PASSWORD '<new>';"
$ sudo nano /srv/intranet/.env # update DATABASE_URL with new password
$ docker compose restart backend
```

4.6 Day-to-day operations cheatsheet

```
# Application containers
$ docker compose ps # show container state
$ docker compose up -d # start (idempotent)
$ docker compose down # stop and remove containers
$ docker compose restart # restart both
$ docker compose restart backend # just the backend
$ docker compose restart web # just nginx
$ docker compose logs -f backend # tail backend logs

# PostgreSQL (native, not in a container)
$ sudo systemctl status postgresql # health
$ sudo systemctl restart postgresql # restart database
$ sudo -u postgres psql intranet_hci # local shell as DB owner
$ sudo journalctl -u postgresql -n 100 # PostgreSQL logs
```

4.7 Database migrations (Prisma)

Schema changes are managed in version control as Prisma migrations under `/srv/intranet/app/prisma/migrations/`. The deployment flow:

- ```
1. Developer edits app/prisma/schema.prisma
 |
 v
2. Locally: npx prisma migrate dev --name <change>
 (creates a new SQL migration file under migrations/)
 |
 v
3. Commit schema.prisma + the new migrations folder
 |
 v
4. Code review + change-management approval
 |
 v
5. On production:
 cd /srv/intranet && docker compose restart backend
 (which runs `prisma migrate deploy` on startup)
```

Manual schema changes via `psql` **bypass this flow and will create drift**. DBAs should treat the schema as code-owned.

## 4.8 Building the frontend

---

```
$ cd /srv/intranet/frontend
$ npm run build # produces fresh dist/
$ npm run lint # ESLint
```

If you forget to rebuild after a frontend edit, the website continues to serve the old UI — the most common source of “I deployed but I don’t see my change.”

## 4.9 Editing the admin or manager portals

---

Plain HTML/JS files:

```
/srv/intranet/public/admin/dashboard.html
/srv/intranet/public/admin/login.html
/srv/intranet/public/manager/index.html
```

Edits are **immediately live** on the next page load; no container restart is needed.

## 4.10 Health and readiness

---

The backend exposes `GET /api/health`:

```
$ curl -sk https://Intranet-HCI.heart.local/api/health
{"ok":true,"db":"connected"}
```

## 4.11 Cleanup recipes

---

```
Stop containers (database untouched)
$ docker compose down

Remove orphaned images / build cache
$ docker system prune -f

Trim journald
$ sudo journalctl --vacuum-time=14d

PostgreSQL maintenance
$ sudo -u postgres vacuumdb --all --analyze
$ sudo -u postgres reindexdb intranet_hci
```

## 4.12 Where to go next

---

- [05-remote-access.md](#) — getting into the box and the DB
- [06-maintenance.md](#) — routine care

## 5 05 — Remote Access

This document covers **all the ways into the production server** from the hospital's internal corporate network — SSH, the Cockpit web console, and the direct database access used by DBAs.

### 5.1 Access matrix

| RHEL 10 VM   |      |                 |                               |
|--------------|------|-----------------|-------------------------------|
|              | Port | Method          | Purpose                       |
| ops jump --> | 22   | SSH (OpenSSH)   | Shell, file transfer, scripts |
|              | 443  | HTTPS (nginx)   | The intranet itself           |
|              | 9090 | HTTPS (Cockpit) | Web-based admin console       |

The PostgreSQL database runs on the SAME VM (native, not in a container):  
Intranet-HCI.heart.local : 5432 (see "Direct database access" below)

There is **no Tailscale, no VPN client, no internet-facing port** for this system. All access is over the hospital's internal corporate network only.

### 5.2 SSH access

From the IT-Ops jump host:

```
$ ssh <ops-user>@Intranet-HCI.heart.local
```

Authentication is **key-based only** in production; password SSH is disabled in `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
PermitRootLogin no
```

To add a new operator's key:

```
$ ssh <ops-user>@Intranet-HCI.heart.local
$ sudo mkdir -p /home/<new-user>/.ssh
$ sudo nano /home/<new-user>/.ssh/authorized_keys
paste the public key
$ sudo chmod 600 /home/<new-user>/.ssh/authorized_keys
$ sudo chown -R <new-user>:<new-user> /home/<new-user>/.ssh
```

## 5.3 Cockpit (web console)

The most beginner-friendly way to manage the server:

```
https://Intranet-HCI.heart.local:9090/
```

Cockpit uses the same TLS certificate as the main site, so browsers show the normal padlock. Log in with the operator's hospital domain account (must be a member of `wheel` group on the VM).

From Cockpit:

```
+-----+
| Cockpit (web) |
|+-----+
| Overview live CPU / memory / disk / network graphs |
| Logs browse journald with search and filters |
| Storage disk partitions, free space, NFS mounts |
| Networking interfaces, firewall, VPN |
| Podman / Docker running containers, restart, view logs |
| Software updates list and install dnf updates with a UI |
| Services systemd unit list, start/stop/enable |
| Accounts list/create users, change passwords |
| Terminal full shell access in the browser |
| SELinux policy state |
|+-----+
+-----+
```

For an operator who is not a Linux expert, **Cockpit is the single best tool to learn**. Most maintenance tasks in `06-maintenance.md` can be performed through Cockpit.

## 5.4 Direct database access

A key production capability: PostgreSQL runs as a **native systemd service** on the same VM as the application (not in a Docker container). This lets DBAs connect directly from their workstations to the same hostname as the website, on the standard PostgreSQL port. There is no need to shell into any container.

### 5.4.1 Database connection details

| Item             | Value                                                             |
|------------------|-------------------------------------------------------------------|
| Hostname         | <code>Intranet-HCI.heart.local</code>                             |
| Port             | <code>5432</code> (default PostgreSQL)                            |
| Database name    | <code>intranet_hci</code>                                         |
| Application user | <code>intranet_app</code> (used by the Fastify backend)           |
| Read-only user   | <code>intranet_ro</code> (for reporting / dashboards)             |
| DBA users        | one per DBA, with full privileges, key-based auth where supported |

### 5.4.2 Connection from a DBA workstation

Using `psql` :

```
$ psql -h Intranet-HCI.heart.local -U <dba-user> -d intranet_hci
```

Using DBeaver, pgAdmin, or another GUI client:

| Field          | Value                                         |
|----------------|-----------------------------------------------|
| Host           | <code>Intranet-HCI.heart.local</code>         |
| Port           | <code>5432</code>                             |
| Database       | <code>intranet_hci</code>                     |
| Username       | the DBA's account                             |
| Authentication | as configured (LDAP / password / certificate) |
| SSL mode       | <code>require</code> (production policy)      |

### 5.4.3 What DBAs can do

- Run ad-hoc `SELECT` queries against any table
- Build reports against the read-only replica
- Take logical backups with `pg_dump`
- Inspect query plans with `EXPLAIN ANALYZE`
- Tune indexes and statistics
- Restore from a backup (with change-management approval)

### 5.4.4 What DBAs should NOT do (without coordination)

- Run `ALTER TABLE` or other schema changes — the application uses **Prisma migrations**; manual changes will drift from the codebase
- Drop or truncate tables on the live DB
- Change column types or constraints without a corresponding code change
- Run long blocking transactions during business hours

Schema changes flow through the codebase:

```

Developer edits prisma/schema.prisma
|
v
npx prisma migrate dev --name <change>
|
v
Commit migration file + schema.prisma
|
v
Code review + change-management approval
|
v
Production deploy applies via `prisma migrate deploy`

```

## 5.5 Quick reference: which tool when

| You want to...              | Use                                                                    |
|-----------------------------|------------------------------------------------------------------------|
| Run a one-line command      | SSH                                                                    |
| Edit a file in vim / nano   | SSH                                                                    |
| Restart a container         | SSH or Cockpit                                                         |
| Watch system metrics        | Cockpit                                                                |
| Read logs with a search box | Cockpit                                                                |
| Update Red Hat packages     | Cockpit (easy) or SSH <code>sudo dnf upgrade</code>                    |
| Query the database          | psql / DBeaver / pgAdmin to <code>Intranet-HCI.heart.local:5432</code> |
| Recover from boot problems  | vSphere Client (vCenter) -> VM -> Web Console                          |

## 5.6 Local console (vSphere)

---

If all remote paths fail, you can still get into the VM via the vSphere Client:

1. Open vCenter at `<vcenter-fqdn>`
2. Log in with your vSphere credentials
3. Locate the VM `Intranet-HCI` in the inventory tree
4. Right-click -> Open Console (or use the embedded HTML5 console)

The console gives you a virtual keyboard/monitor connection to the VM, useful when networking or SSH is broken.

## 5.7 Where to go next

---

- `06-maintenance.md` — routine maintenance procedures
- `07-troubleshooting.md` — when something is broken

## 6 06 — Maintenance Guide

*This chapter is written for people new to Red Hat Linux. It explains every command and what each piece does. If you are an experienced sysadmin, skim the code blocks; the prose is for newcomers.*

If you remember nothing else: **most problems are solved by restarting the right service**. Identify which layer is broken (web, backend, database, OS), then restart just that piece.

### 6.1 1. Orientation — what you're looking at

When you log in via SSH, you are dropped into a **shell** (the program that reads commands and runs them). The default shell here is `bash`. The prompt looks like:

```
bryant@localhost:~$
```

- "I'm waiting for a command"
- you are in your home folder (~/)
- the machine's hostname
- your username

The most useful keys at the prompt:

| Key                 | What it does                                                      |
|---------------------|-------------------------------------------------------------------|
| <code>Tab</code>    | autocompletes file names and commands                             |
| <code>↑ / ↓</code>  | cycles through command history                                    |
| <code>Ctrl+R</code> | searches command history (start typing, <code>↑</code> to scroll) |
| <code>Ctrl+C</code> | cancels the currently running command                             |
| <code>Ctrl+L</code> | clears the screen (same as typing <code>clear</code> )            |
| <code>Ctrl+D</code> | logs out (same as <code>exit</code> )                             |

#### 6.1.1 `sudo` — running things as root

You are logged in as `bryant`. For commands that change the system (installing software, restarting services, editing system files), prefix the command with `sudo`. The first time per session it asks for your password. Once you've entered it, subsequent `sudo` calls in the same shell skip the prompt for a few minutes.

```
$ whoami # → bryant
$ sudo whoami # → root (after entering password)
```

⚠ `sudo` is a loaded gun — it runs commands with full system privileges. Always read the command before pressing Enter.

## 6.2 2. Daily / weekly checks

### 6.2.1 Is the website up?

```
$ curl -sI http://localhost:8080/ # should print "HTTP/1.1 200 OK"
$ curl -s http://localhost:8080/api/health
{"ok":true}
```

If either fails, jump to `07-troubleshooting.md` → *Website is down*.

### 6.2.2 Are the containers running?

```
$ cd /srv/intranet
$ docker compose ps
```

You should see both `intranet_nginx` and `intranet_backend` with status `Up`. Anything other than `Up` is a problem.

### 6.2.3 How is disk space?

```
$ df -h / # root filesystem free space
$ du -sh /srv/intranet/uploads/ # how much uploads have grown
$ docker system df # Docker's view of disk usage
```

Treat **anything above 80% used** as urgent. Above 90% the OS misbehaves in subtle ways (the Docker daemon can refuse to start new containers, journald truncates logs, the website may fail to serve uploads).

### 6.2.4 How is memory?

```
$ free -h
```

You expect about 3–4 GiB used. If `available` drops below 500 MiB, something is leaking or a workload spiked. The `top` or `htop` commands let you see which process is to blame:

```
$ top # press 'q' to quit, 'M' to sort by memory, 'P' by CPU
```

💡 In Cockpit, the **Overview** page shows all of this graphically — no commands needed.

### 6.3 3. Restarting things (the most common operation)

The intranet has several restart layers, from least to most disruptive:

```
Restart NGINX only (5 sec downtime)
 ↓
Restart Backend only (15–30 sec downtime)
 ↓
Restart Both (full intranet downtime, ~30 sec)
 ↓
Restart Docker daemon (~1 minute, restarts ALL containers)
 ↓
Reboot the VM (3–5 minutes total)
```

Run all of these from `/srv/intranet/`:

```
$ cd /srv/intranet
$ docker compose restart web # just nginx
$ docker compose restart backend # just the Node app
$ docker compose restart # both
$ sudo systemctl restart docker # whole Docker daemon (RARE)
$ sudo systemctl reboot # full VM reboot
```

⚠️ **Reboot only when you need to** — the website is offline for 3–5 minutes, and any running maintenance tasks die.

### 6.4 4. Reading logs

Logs are the first thing to look at when something is wrong.

#### 6.4.1 Application logs (the containers)

```
$ cd /srv/intranet
$ docker compose logs --tail=100 backend # last 100 lines, backend
$ docker compose logs --tail=100 web # nginx
$ docker compose logs -f backend # live tail (Ctrl+C to stop)
$ docker compose logs --since 1h backend # past hour
```

What “good” backend logs look like:

```
intranet_backend | Server listening on :3000
intranet_backend | [email stub] subject → recipient
```

What “bad” looks like:

```
intranet_backend | Error: P1001 Can't reach database server at host.docker.internal:
5432
intranet_backend | UnhandledPromiseRejection ...
```

## 6.4.2 System logs (anything outside the containers)

The system uses `journald` — a unified log service. Useful commands:

```
$ sudo journalctl -xe # last entries, formatted
$ sudo journalctl --since "1 hour ago" # past hour
$ sudo journalctl --since today # since midnight
$ sudo journalctl -u docker.service # logs for a specific service
$ sudo journalctl -u sshd # SSH logs
$ sudo journalctl -k # kernel messages only
$ sudo journalctl -f # live tail of everything
```

 In Cockpit, **Logs** in the left navigation gives a search-and-filter UI for exactly this.

## 6.5 5. Updating the operating system

Red Hat publishes security and feature updates regularly. We are *registered* with the Red Hat subscription system, so `dnf` (the package manager) can pull them.

```
$ sudo subscription-manager status # confirm we are registered
$ sudo dnf check-update # list available updates (no changes)
$ sudo dnf upgrade # apply updates (asks "y/n")
$ sudo dnf upgrade -y # same, but no prompt
```

After installing updates, **check if a reboot is needed**:

```
$ sudo dnf needs-restarting -r # exits 1 if reboot recommended
```

If a reboot is needed (kernel, glibc, systemd updates), schedule it:

```
$ sudo systemctl reboot
```

### 6.5.1 Recommended update cadence

| Cadence   | Action                                                                   |
|-----------|--------------------------------------------------------------------------|
| Weekly    | <code>sudo dnf check-update</code> — read the list, decide what to apply |
| Monthly   | <code>sudo dnf upgrade --security</code> — security-only updates         |
| Quarterly | Full <code>sudo dnf upgrade</code> + reboot, during a maintenance window |

⚠ **Never run `dnf upgrade` mid-workday.** Some packages restart services mid-update; the intranet may briefly stop responding.

### 6.5.2 Cockpit alternative

Cockpit's **Software Updates** page lets you click-through updates with a human-readable changelog, and it warns you when a reboot is required.

## 6.6 6. Backups

### 6.6.1 What needs backing up

Four things, in priority order:

| Importance  | Path                                                                          | Why                                                       |
|-------------|-------------------------------------------------------------------------------|-----------------------------------------------------------|
| ● Critical  | PostgreSQL <code>intranet_hci</code> database                                 | Source of truth for all relational data                   |
| ● Critical  | <code>/srv/intranet/uploads/</code>                                           | All uploaded photos/files                                 |
| ● Important | <code>/srv/intranet/.env</code>                                               | <code>SESSION_SECRET</code> and <code>DATABASE_URL</code> |
| ● Important | <code>/var/lib/pgsql/data/postgresql.conf</code> and <code>pg_hba.conf</code> | DB configs needed to rebuild                              |
| ● Nice      | <code>/srv/intranet/app/</code> , <code>/srv/intranet/frontend/</code>        | Source code (also in git)                                 |

## 6.6.2 Manual backup recipe

```
$ DATE=$(date +%Y%m%d-%H%M%S)

1. Dump the PostgreSQL database (safe to run live; uses MVCC snapshot)
$ sudo -u postgres pg_dump -Fc intranet_hci \
 > /home/<ops-user>/intranet-db-$DATE.dump

2. Tar the uploads
$ sudo tar -czf /home/<ops-user>/intranet-uploads-$DATE.tar.gz \
 -C /srv intranet/uploads

3. Copy the .env (small, but contains secrets)
$ sudo cp /srv/intranet/.env /home/<ops-user>/intranet-env-$DATE.backup
```

Then **copy the backups off the VM** (this is the part people forget):

```
From an operator workstation:
$ scp <ops-user>@Intranet-HCI.heart.local:/home/<ops-user>/intranet-*-$DATE.* \
 ~/Backups/intranet/
```

## 6.6.3 Why pg\_dump is safe to run live

`pg_dump` uses PostgreSQL's MVCC snapshot mechanism: it takes a consistent point-in-time view of all tables, and the backend can continue reading and writing concurrently. No need to stop the application for the dump.

## 6.7 Hot vs cold backup

⚠ The PostgreSQL database is being **actively written** by the backend. A `tar` of the `intranet_hci` database could capture it mid-write and produce a slightly inconsistent file. For a truly clean backup:

```
$ sudo -u postgres pg_dump -Fc intranet_hci > /var/backups/intranet/intranet-db-$(date +%Y%m%d-%H%M%S).dump
$ sudo tar -czf /home/bryant/intranet-backup-$DATE.tar.gz \
 -C /srv intranet/data/intranet-backup.db intranet/uploads intranet/.env
$ sudo rm /srv/intranet/data/intranet-backup.db
```

`.backup` is PostgreSQL's online backup API — safe to run while the app is live.

### 6.7.1 Automated backups (recommended)

Save the following as `/usr/local/bin/intranet-backup.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
BACKUP_DIR=/var/backups/intranet
DATE=$(date +%Y%m%d-%H%M%S)
mkdir -p "$BACKUP_DIR"

PostgreSQL logical dump (compressed custom format)
sudo -u postgres pg_dump -Fc intranet_hci \
 > "$BACKUP_DIR/intranet-db-$DATE.dump"

Uploads
tar -czf "$BACKUP_DIR/intranet-uploads-$DATE.tar.gz" \
 -C /srv intranet/uploads

Symlink "LATEST" for convenience
ln -sf "$BACKUP_DIR/intranet-db-$DATE.dump" "$BACKUP_DIR/intranet-db-
LATEST.dump"
ln -sf "$BACKUP_DIR/intranet-uploads-$DATE.tar.gz" "$BACKUP_DIR/intranet-uploads-
LATEST.tar.gz"

Keep only the last 14 backups of each type
ls -lt "$BACKUP_DIR"/intranet-db-*.dump | tail -n +15 | xargs -r rm --
ls -lt "$BACKUP_DIR"/intranet-uploads-*.tar.gz | tail -n +15 | xargs -r rm --
```

Then schedule it nightly via cron:

```
$ sudo crontab -e
Add this line:
0 2 * * * /usr/local/bin/intranet-backup.sh >> /var/log/intranet-backup.log 2>&1
```

This runs at 2:00 a.m. every night and keeps two weeks of snapshots.

**⚠ Still copy them off the VM.** Backups stored only on the VM do not help if the VM is the thing that fails. Sync nightly to a hospital-managed backup share or cloud bucket.

## 6.8 7. Disk space management

If the disk fills up, the system breaks in confusing ways. Routine cleanup:

### 6.8.1 What's eating the disk?

```
$ df -h / # overall free space
$ sudo du -sh /var/* 2>/dev/null | sort -h # biggest /var/ subdirs
$ sudo du -sh /home/* 2>/dev/null | sort -h
$ docker system df # Docker's footprint
$ sudo journalctl --disk-usage # log size
```

## 6.8.2 Common cleanup commands

```
Old Docker images and build cache
$ docker system prune -a
$ docker container prune

Trim journald logs to last 14 days
$ sudo journalctl --vacuum-time=14d

Trim journald to a size cap
$ sudo journalctl --vacuum-size=500M

DNF cache
$ sudo dnf clean all
```

```
⚠ removes UNUSED images
removes stopped containers
```

⚠ The `intranet-backup.tar.gz` file in `/srv/intranet/` is 48 MB and dates from May 15. Decide whether to keep it or move it to `/home/bryant/`.

## 6.9 8. Managing users

Adding a Linux user (e.g., for another technical operator):

```
$ sudo useradd -m -G wheel,docker alice
creates home dir, gives sudo and docker access

$ sudo passwd alice
$ sudo passwd -e alice
login
```

```
sets a password
forces password change on first
```

`wheel` is Red Hat's "may use sudo" group. `docker` lets the user run `docker` commands without sudo.

Removing a user:

```
$ sudo userdel -r alice
```

```
-r removes their home directory
```

⚠ **Application users (admin accounts inside the intranet website) are different** — those live in the `users` table of the `intranet_hci` database and are managed through the website's admin panel at `/admin/`, not via Linux.

## 6.10 9. Changing the intranet admin password

If you forgot the admin password for the website (not the Linux account — the website admin user inside the application):

```
1. Generate a new bcrypt hash inside the backend container
$ docker compose exec backend node -e \
 "console.log(require('bcryptjs').hashSync('NewPass123!', 10))"
Copy the resulting $2a$10$... string

2. Update the user row in PostgreSQL
$ sudo -u postgres psql intranet_hci -c \
 "UPDATE \"User\" SET password_hash = '<paste-hash-here>' WHERE username =
 'admin';"
```

Or, if you prefer GUI tools (DBBeaver, pgAdmin), update the same row through your DBA workstation — connect to `Intranet-HCI.heart.local:5432` as a DBA-privileged user, navigate to the `User` table, edit the `password_hash` column.

Log in with the new password.

---

## 6.11 10. SSL/TLS — should we do this?

Currently the intranet is served over **plain HTTP** on port 8080. On a trusted LAN this is the historical norm, but it has weaknesses:

- Passwords cross the network in cleartext
- Browsers warn about “Not Secure”
- Session cookies cannot be marked `Secure: true`

Adding TLS is a recommended hardening step. Two paths:

1. **Self-signed certificate** for nginx — simple, browsers will warn
2. **Reverse-proxy through Caddy or Traefik** with a Let’s Encrypt cert via DNS-01 (works for LAN-only services)

This is out of scope for routine maintenance but worth knowing about.

---

## 6.12 11. Routine maintenance schedule

A reasonable rhythm for this system:

```
DAILY (5 min, automated)
├─ Backup script runs at 02:00
└─ (Optional) cron health-check pinging /api/health

WEEKLY (10 min, manual)
├─ docker compose ps → containers all Up
├─ df -h / → disk under 80%
├─ sudo journalctl -p err --since=7d → any errors?
└─ Check the corporate network admin console → all expected devices online

MONTHLY (30 min, manual, maintenance window)
├─ sudo dnf upgrade --security → security updates
├─ Review backup integrity (extract one and check files)
└─ Rotate logs if growing fast

QUARTERLY (1 hour, manual, maintenance window)
├─ Full sudo dnf upgrade + reboot
├─ Restart all containers from cold
├─ Verify backups by performing a TEST RESTORE
└─ Review user list, deactivate inactive accounts
```

## 6.13 12. When in doubt

Three commands that have saved many sessions:

```
$ cd /srv/intranet && docker compose restart # nudge the app
$ sudo systemctl reboot # nudge the OS
$ sudo journalctl -xe # what just broke?
```

If those don't help, see [07-troubleshooting.md](#).

## 7 07 — Troubleshooting

---

This is a **playbook of known problems and their fixes**. When something is broken, find the matching symptom and follow the steps.

### 7.1 How to use this document

---

1. Identify the layer that's broken:
  - Can you SSH in?
  - Does the website respond at all?
  - Does /api/health work?
  - Do API calls return data?
2. Look for matching symptom below
3. Follow the playbook

- OS is up
- nginx is up
- backend is up
- database is reachable

### 7.2 Symptom index

---

- Website is completely down
- Website loads but shows error / blank page
- API calls return 401 / "Not logged in"
- Login keeps failing
- Uploaded files don't appear
- RDP connection hangs at "connecting"
- RDP error 0x207 (Windows App)
- SSH refuses connection
- Disk is full
- Containers keep restarting
- The database is locked
- I forgot the admin password
- SELinux is blocking something
- GPG check failed during dnf upgrade

### 7.3 Website is completely down

---

**Symptom:** Browser cannot reach `https://Intranet-HCI.heart.local/`. Connection refused, or times out.

**Triage:**

```
$ ssh bryant@Intranet-HCI.heart.local # can you log in?
$ ping <vsphere-assigned-ip> # is the VM on the network?
```

If you cannot ping or SSH, the VM is down or off the network. Check vSphere.

If you can SSH, continue:

```
$ cd /srv/intranet
$ docker compose ps # are containers Up?
$ sudo ss -tlnp | grep 8080 # is port 8080 listening?
$ curl -I http://localhost:8080/ # local probe
```

### Common causes and fixes:

1. **Containers stopped** — `docker compose up -d`
2. **Backend in restart loop** — `docker compose logs --tail=200 backend` reveals why. Usually `SESSION_SECRET` missing or a syntax error in `server.js`.
3. **nginx misconfig** — `docker compose logs --tail=50 web` shows nginx errors. Look for “host not found in upstream” → means backend died.
4. **Firewall blocking port 8080** — `sudo firewall-cmd --list-all` should show `ports: 3389/tcp 8080/tcp ...`. Re-add if missing:

```
$ sudo firewall-cmd --permanent --add-port=8080/tcp
$ sudo firewall-cmd --reload
```

5. **Disk full** — `df -h /`. If above 95%, see [Disk is full](#).

---

## 7.4 Website loads but shows error / blank page

---

**Symptom:** Browser reaches the site but shows a blank screen, JavaScript error, or HTTP 502 / 504 from nginx.

### Triage:

```
$ curl -i http://localhost:8080/api/health # backend alive?
$ docker compose logs --tail=100 backend
```

**HTTP 502 Bad Gateway:** nginx can reach the backend container but the backend isn’t responding. Restart it:

```
$ docker compose restart backend
$ docker compose logs -f backend
```

**HTTP 504 Gateway Timeout:** backend is slow. Check load:

```
$ top # is something pegged at 100% CPU?
$ docker stats # container-level CPU/memory
```

**Blank page in browser:** the React build is stale or broken. Rebuild:

```
$ cd /srv/intranet/frontend
$ npm run build
```

---

## 7.5 API calls return 401 / “Not logged in”

---

**Symptom:** You can see the site, but most API calls return 401.

**Cause:** Your session cookie expired or the `SESSION_SECRET` changed.

**Fix:** Log out and back in. If that fails for everyone, check that the backend started with the secret:

```
$ docker compose logs backend | grep -i secret
Should NOT see "FATAL: SESSION_SECRET environment variable is not set"
```

If the backend can't read `.env` :

```
$ ls -l /srv/intranet/.env
$ cat /srv/intranet/.env # confirm a value exists
```

---

## 7.6 Login keeps failing

---

**Symptom:** correct username/password but login fails.

**Possible causes:**

1. **Rate limited.** After 20 failed attempts in 15 minutes, the IP is blocked. Wait or restart the backend:

```
$ docker compose restart backend
```

2. **Wrong password (it happens).** Reset it — see [I forgot the admin password](#).

3. **Backend can't write to session table.** Check disk space and permissions:

```
$ df -h /srv
$ ls -la /srv/intranet/data/
```

## 7.7 Uploaded files don't appear

**Symptom:** files upload “successfully” but 404 when opened.

**Triage:**

```
$ ls -la /srv/intranet/uploads/ # is the file there?
$ docker compose exec backend ls -la /uploads/ # visible inside container?
$ curl -I http://localhost:8080/files/<filename> # nginx → backend → static
```

**Common cause:** the `/uploads` mount got disconnected after a Docker change. Recreate containers:

```
$ cd /srv/intranet
$ docker compose down
$ docker compose up -d
```

## 7.8 RDP connection hangs at “connecting”

**Symptom:** Royal TSX (or other client) shows “Connecting...” indefinitely.

**Triage:**

```
$ ss -tlnp | grep 3389 # is it listening?
$ sudo systemctl status gnome-remote-desktop # is the service up?
$ sudo journalctl -u gnome-remote-desktop -n 50 # recent logs
```

**Common fix:**

```
$ sudo systemctl restart gnome-remote-desktop
```

**If credentials are empty in `grdctl status` :**

```
$ sudo grdctl --system rdp set-credentials bryant '<password>'
$ sudo systemctl restart gnome-remote-desktop
```

**If the TLS cert/key are gone:**

```
$ sudo openssl req -new -x509 -days 3650 -nodes \
 -out /etc/gnome-remote-desktop/rdp-tls.crt \
 -keyout /etc/gnome-remote-desktop/rdp-tls.key \
 -subj "/CN=rhel-vm"
$ sudo grdctl --system rdp set-tls-cert /etc/gnome-remote-desktop/rdp-tls.crt
$ sudo grdctl --system rdp set-tls-key /etc/gnome-remote-desktop/rdp-tls.key
$ sudo systemctl restart gnome-remote-desktop
```

---

## 7.9 RDP error 0x207 (Windows App)

---

**Symptom:** Microsoft's "Windows App" on Mac shows error `0x207` and disconnects immediately.

**Cause:** Headless gnome-remote-desktop uses **RDP server redirection**, which Microsoft's Windows App on macOS does not honor.

**Fix:** Use a different client. Royal TSX (free) handles redirection correctly. See `05-remote-access.md` → *Recommended client*.

---

## 7.10 SSH refuses connection

---

**Symptom:** `ssh: connect to host ... port 22: Connection refused`

**Most likely:** `sshd` is not running.

From the vSphere console (or any working remote path):

```
$ sudo systemctl status sshd
$ sudo systemctl start sshd
$ sudo systemctl enable sshd
```

**If you only have an SSH problem from outside the LAN:**

- Check the corporate network: `sudo systemctl status (none)`
  - Check firewall: `sudo firewall-cmd --list-ports` should include `22/tcp` (or via `services: ssh`)
- 

## 7.11 Disk is full

---

**Symptom:** Random operations fail with "No space left on device." `df -h /` shows above 95% used.

**Quick relief (in order, run as you go):**

```
1. Old Docker images and build cache
$ docker system prune -a -f

2. journald logs over 14 days
$ sudo journalctl --vacuum-time=14d

3. DNF cache
$ sudo dnf clean all

4. Anything weird in /tmp
$ sudo du -sh /tmp/*
$ sudo rm -rf /tmp/<stale-thing>

5. The 48 MB backup tarball at /srv/intranet/intranet-backup.tar.gz
$ ls -lh /srv/intranet/intranet-backup.tar.gz
Move it to your home or delete:
$ mv /srv/intranet/intranet-backup.tar.gz /home/bryant/
```

If you cannot find what filled the disk, this command lists the biggest directories anywhere:

```
$ sudo du -h --max-depth=2 / 2>/dev/null | sort -rh | head -30
```

**If `/srv/intranet/uploads/` is the problem:** users have uploaded too much. Decide what to prune (an admin can delete posts/resources through the website). Do not delete files from `/srv/intranet/uploads/` directly — the database references them by filename.

---

## 7.12 Containers keep restarting

---

**Symptom:** `docker compose ps` shows `Restarting` or repeatedly cycling between `Up` and `Restarting`.

**Triage:**

```
$ docker compose logs --tail=200 <service>
```

Look for the very first error after each restart. Most likely:

- **Backend:** `SESSION_SECRET` missing or PostgreSQL unreachable
  - **Backend:** a recent code change has a syntax error
  - **nginx:** `host not found in upstream "backend"` → backend isn't running
  - **nginx:** typo in `/srv/intranet/nginx/default.conf` → test with `docker compose exec web nginx -t`
-

## 7.13 Database errors in the backend logs

**Symptom:** Backend logs show PostgreSQL connection errors:

P1001: Can't reach database server , P2024: Timed out fetching a connection from the pool , or sustained pool-exhaustion warnings.

**Triage:**

```
$ sudo systemctl status postgresql # database service up?
$ ss -tlnp | grep 5432 # listening?
$ docker compose exec backend sh -c \
 "nc -zv host.docker.internal 5432" # reachable from container?
$ sudo journalctl -u postgresql -n 50 # postgres log tail
```

**Common causes and fixes:**

- **PostgreSQL crashed or didn't start** — `sudo systemctl restart postgresql`
- **Connection limit reached** — check active connections: `sudo -u postgres psql -c "SELECT count(*) FROM pg_stat_activity;"`. If near `max_connections`, restart the backend to release pooled connections, then consider raising the limit in `postgresql.conf`.
- **Wrong password in `.env`** — `DATABASE_URL` mismatch with what PostgreSQL expects. Compare to the role's actual password.
- **`pg_hba.conf` rejecting the connection** — the postgres journal will say "no pg\_hba.conf entry" or "password authentication failed".

## 7.14 I forgot the admin password

**The Linux admin password** (the `bryant` account):

1. From the vSphere Web Console (vCenter) console (you'll need physical or hypervisor access), reboot the VM
2. At the GRUB menu, press `e` to edit the boot entry
3. Find the line starting with `linux ...`, add `rd.break enforcing=0` to the end
4. Ctrl-X to boot
5. At the `switch_root:/#` prompt:

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd
exit
exit
```

**The application admin password** (the `admin` user in the website): See `06-maintenance.md` → *Changing the intranet admin password.*

---

## 7.15 SELinux is blocking something

---

**Symptom:** A service “should work” but doesn’t, and `journalctl` mentions “AVC denied” or `setroubleshoot`.

**Check:**

```
$ getenforce # current mode (Enforcing/Permissive)
$ sudo ausearch -m AVC --start recent # recent denials
```

**Quick test:** temporarily set to permissive to confirm it’s SELinux:

```
$ sudo setenforce 0 # permissive – NOT persistent
test the failing operation
$ sudo setenforce 1 # back to enforcing
```

If SELinux is the culprit, the **proper** fix is to label files correctly or generate a custom policy with `audit2allow`. The lazy fix (don’t use long-term) is to set permissive mode in `/etc/selinux/config`.

⚠ Disabling SELinux reduces your security posture. Treat it as a diagnostic step, not a fix.

---

## 7.16 GPG check failed during dnf upgrade

---

**Symptom:** `dnf upgrade` shows messages like:

```
GPG Keys are configured as: file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
The downloaded packages were saved in cache until the next successful transaction.
Error: GPG check FAILED
```

**Cause:** A repository pushed a package signed by a key the system doesn’t trust yet (often happens after enabling EPEL or third-party repos with newer keys).

**Fix options:**

1. Refresh the keys:

```
$ sudo dnf clean all
$ sudo subscription-manager refresh
$ sudo dnf upgrade
```

2. If a specific third-party repo (EPEL, etc.) is the culprit, disable it for the upgrade:

```
$ sudo dnf upgrade --disablerepo=epel
```

3. As a last resort, install the new key:

```
$ sudo rpm --import https://path/to/the/repo/KEY
```

---

## 7.17 When all else fails

---

Step back and run the **golden three**:

```
$ cd /srv/intranet && docker compose down && docker compose up -d
$ sudo systemctl reboot
$ sudo journalctl -xe --since "10 minutes ago"
```

If the system still won't come up, restore from backup ( [08-disaster-recovery.md](#) ).

## 8 08 — Disaster Recovery

---

This document covers **what to do when something is broken badly enough that routine maintenance can't fix it**. The procedures here assume you have backups (see `06-maintenance.md` → *Backups*).

### 8.1 Recovery scenarios

---

| Scenario                                                      | First step                                              |
|---------------------------------------------------------------|---------------------------------------------------------|
| -----                                                         | -----                                                   |
| Database corruption                                           | Restore database from pg_dump                           |
| Files deleted from /uploads/<br>/srv/intranet/ wiped entirely | Restore uploads from backup<br>Full application restore |
| Whole VM unbootable                                           | Roll back from vSphere snapshot                         |
| Hardware failure on vSphere host                              | Restore VM image to another host                        |

### 8.2 Scenario 1 — Database is corrupted

---

**Symptom:** Backend keeps crashing with PostgreSQL errors, or a query like `psql -d intranet_hci -c "\dt"` returns connection or relation errors that shouldn't be possible.

**Recovery steps:**

```
1. Stop the backend so it stops writing
$ cd /srv/intranet
$ docker compose stop backend

2. Confirm PostgreSQL itself is still running on the host
$ sudo systemctl status postgresql

3. Take a snapshot of the bad state for forensics (do not skip)
$ sudo -u postgres pg_dump -Fc intranet_hci > \
 /var/backups/intranet/intranet-BROKEN-$(date +%Y%m%d-%H%M%S).dump

4. Find your latest good backup
$ ls -lh /var/backups/intranet/intranet-*.dump

5. Drop the corrupted database and restore from backup
$ sudo -u postgres psql -c "DROP DATABASE intranet_hci;"
$ sudo -u postgres psql -c "CREATE DATABASE intranet_hci OWNER intranet_app;"
$ sudo -u postgres pg_restore --no-owner --role=intranet_app \
 -d intranet_hci /var/backups/intranet/intranet-YYYYMMDD-HHMMSS.dump

6. Restart and verify
$ docker compose start backend
$ docker compose logs -f backend # watch for errors
$ curl -sk https://Intranet-HCI.heart.local/api/health
```

 Restoring the DB rolls back to the backup time. Any posts, signups, or nominations made after the backup are **lost**. Communicate to users.

## 8.3 Scenario 2 — Uploads directory missing or wiped

**Symptom:** Images show as broken; `/srv/intranet/uploads/` is empty or missing files.

**Recovery:**

```
$ docker compose stop backend # avoid mid-write

Extract uploads from the backup
$ sudo tar -xzf /var/backups/intranet/intranet-uploads-LATEST.tar.gz \
 -C /srv/intranet/uploads # restores in place

Verify
$ ls /srv/intranet/uploads/ | head
$ docker compose start backend
```

If only **some** files are missing (a user deleted a folder), extract the backup to a temp directory and copy in only what you need:

```
$ mkdir /tmp/restore
$ sudo tar -xzf /var/backups/intranet/intranet-uploads-LATEST.tar.gz -C /tmp/restore
$ ls /tmp/restore/
$ sudo cp /tmp/restore/<file> /srv/intranet/uploads/
$ sudo rm -rf /tmp/restore
```

## 8.4 Scenario 3 — Whole `/srv/intranet/` is gone

**Symptom:** Application files deleted, project directory empty, or ransomware-like situation.

**Recovery:**

```
1. Re-clone source code from the internal git repository
$ sudo mkdir -p /srv/intranet
$ sudo chown <ops-user>:<ops-user> /srv/intranet
$ cd /srv
$ git clone https://<internal-git>/heart-center/intranet.git

2. Restore uploads from backup
$ sudo tar -xzf /var/backups/intranet/intranet-uploads-LATEST.tar.gz \
 -C /srv/intranet/uploads

3. Re-create the .env file
$ openssl rand -hex 48 # generate SESSION_SECRET
$ sudo nano /srv/intranet/.env
Paste these lines, filling in real values:
SESSION_SECRET=<hex from openssl>
DATABASE_URL=postgresql://intranet_app:<pw>@host.docker.internal:5432/
intranet_hci
INITIAL_ADMIN_PASSWORD=<set if seeding fresh>

4. The database is intact (it lives in /var/lib/pgsql/data and is
not affected when /srv/intranet/ is wiped). If it is also gone,
follow Scenario 1 first.

5. Build the frontend
$ cd /srv/intranet/frontend
$ npm install
$ npm run build

6. Bring the containers back up
$ cd /srv/intranet
$ docker compose up -d
$ docker compose logs -f
```

## 8.5 Scenario 4 — PostgreSQL data directory is corrupted

**Symptom:** `systemctl status postgresql` shows the service failed to start with errors about WAL files, missing tablespaces, or page checksum failures.

**Recovery:**

```
1. Stop PostgreSQL
$ sudo systemctl stop postgresql

2. Preserve the broken data dir for forensics
$ sudo mv /var/lib/pgsql/data /var/lib/pgsql/data.broken-$(date +%Y%m%d)

3. Initialize a fresh data dir
$ sudo /usr/bin/postgresql-setup --initdb

4. Restore the configs from your config backup or re-run the steps in
04-deployment.md to recreate postgresql.conf, pg_hba.conf, users.
$ sudo systemctl start postgresql

5. Recreate database and restore from pg_dump
$ sudo -u postgres psql -c "CREATE DATABASE intranet_hci OWNER intranet_app;"
$ sudo -u postgres pg_restore --no-owner --role=intranet_app \
 -d intranet_hci /var/backups/intranet/intranet-LATEST.dump

6. Restart the backend
$ cd /srv/intranet && docker compose restart backend
```

## 8.6 Scenario 5 — VM won't boot

**Symptom:** VM hangs at boot, kernel panic, root filesystem unmountable.

This is where **vSphere snapshots** save you. If snapshots were taken before the trouble started:

1. Open the vSphere Client (vCenter)
2. Locate VM **Intranet-HCI** in the inventory
3. Right-click → Snapshots → Manage Snapshots
4. Select the most recent good snapshot → **Revert**
5. The VM reverts to that point in time

 Reverting to a snapshot discards everything that happened after it, including database rows and uploaded files. **Combine with a recent backup restore** (Scenarios 1 and 2) to get back the latest data.

### 8.6.1 If no snapshots exist

If the VM disk is intact but the OS won't boot:

1. From vSphere, attach the RHEL boot ISO to the VM
2. Power-cycle the VM and boot to "Rescue installed system"
3. The installer mounts your existing root at **/mnt/sysimage**
4. **chroot /mnt/sysimage**

5. Diagnose: `journalctl -xb` for the last boot, `dnf history` for recent package operations, check `/etc/fstab` for mount typos
6. `exit` , detach the ISO, reboot

## 8.7 Scenario 6 — Total hypervisor loss

**Symptom:** The vSphere host (or the whole cluster) is gone — fire, theft, storage failure beyond recovery.

This is the worst case. Recovery depends on what you have off-site:

| Off-site asset                       | Restore path                                                                                       |
|--------------------------------------|----------------------------------------------------------------------------------------------------|
| Tar/PGDump backups (recent) on a NAS | Provision a new RHEL VM, run through <code>04-deployment.md</code> , restore data (Scenarios 1, 2) |
| vSphere snapshot exported as OVA     | Import the OVA into another vSphere cluster                                                        |
| Only the pg_dump + uploads tar       | Build a fresh RHEL VM, install Docker + PostgreSQL, restore                                        |
| Nothing                              | Rebuild from scratch using this documentation as the guide                                         |

The lesson: **always have at least the PostgreSQL dumps and uploads tar stored off the vSphere cluster.** Hospital backup policy should already cover this; verify with the backup administrator.

## 8.8 Recovery time objectives (informational)

These are **estimates** for an experienced operator with backups available:

| Scenario                            | Estimated downtime              |
|-------------------------------------|---------------------------------|
| Database corruption only            | 10–20 minutes                   |
| Uploads only                        | 10–30 minutes (depends on size) |
| <code>/srv/intranet/</code> wiped   | 30–60 minutes                   |
| Postgres data dir corrupted         | 20–45 minutes                   |
| VM unbootable, snapshot available   | 5–15 minutes                    |
| VM unbootable, no snapshot          | 1–3 hours                       |
| Total hypervisor loss, full rebuild | 4–8 hours                       |

## 8.9 Testing your recovery procedure

---

 **A backup you have never restored is not a backup.** Test once a quarter:

```
1. Create a scratch database
$ sudo -u postgres psql -c "CREATE DATABASE intranet_hci_test;"

2. Restore your latest pg_dump into it
$ sudo -u postgres pg_restore --no-owner --role=intranet_app \
 -d intranet_hci_test /var/backups/intranet/intranet-LATEST.dump

3. Sanity-check row counts
$ sudo -u postgres psql -d intranet_hci_test -c \
 "SELECT 'users', COUNT(*) FROM users
 UNION ALL
 SELECT 'posts', COUNT(*) FROM posts;"

4. Drop the test database
$ sudo -u postgres psql -c "DROP DATABASE intranet_hci_test;"

5. Spot-check uploads tarball
$ tar -tzf /var/backups/intranet/intranet-uploads-LATEST.tar.gz | head
```

If any step fails, **your backups are not viable** and you need to fix the backup procedure before something real breaks.

## 8.10 Building a runbook for your specific environment

---

Once a year, sit down and write a one-page “if I had to do this from scratch today” document:

- Where do the backups live? (Path, hostname, credentials)
- Who is the responsible IT-Operations point of contact?
- Where is the vSphere / vCenter login? (URL, credentials in the password manager)
- Where is the Red Hat subscription tied to? (Account, contact email)
- Who is the database administrator on call?
- Who else knows? (Designate a backup operator)

Store this off-site too. Print a copy. The day you need it, you will not have internet access.

## 9 09 — Glossary

---

A reference for the terms used throughout this documentation. Skim it once, then refer back as needed.

### 9.1 Networking

---

**LAN** (Local Area Network) — the practice's internal network, on the Huntsville Hospital corporate VLAN. Devices on the LAN can reach the intranet directly.

**Reverse proxy** — a server that accepts requests on behalf of another. Here, nginx accepts HTTPS requests on port 443 and forwards them to the Fastify backend on port 3000.

**Docker bridge network** — a virtual switch that lets containers talk to each other by name. The intranet uses `intranet_default`.

**Docker bridge gateway** — the host-side endpoint of the Docker bridge (typically `172.18.0.1`). Containers reach the host's networking through this address. The Fastify backend uses it to reach PostgreSQL on the host.

`host.docker.internal` — a hostname Docker can be configured to provide inside containers, resolving to the host machine. The backend's `DATABASE_URL` uses this to find PostgreSQL.

**Loopback** — `127.0.0.1`, the address that always means "this machine."

### 9.2 Operating system

---

**RHEL** — Red Hat Enterprise Linux. The OS this VM runs.

**dnf** — RHEL's package manager. Equivalent to `apt` on Debian/Ubuntu.

**rpm** — the low-level package format used by dnf. You will rarely call `rpm` directly.

**systemd** — RHEL's init system and service manager. Started at boot, runs everything else (PostgreSQL, Docker, sshd, etc.).

**systemctl** — the command to control systemd units (start, stop, enable, status).

**Unit / service** — a thing systemd manages. Lives in `/usr/lib/systemd/` or `/etc/systemd/`.

**journald / journalctl** — systemd's log collector and the tool to query it.

**SELinux** — Red Hat's mandatory access control system. Adds an extra layer of security on top of file permissions. Modes: `Enforcing`, `Permissive`, `Disabled`.

**firewalld** — RHEL's firewall manager. Wraps `nftables`.

**SSH** — Secure Shell. The encrypted remote-login protocol on port 22.

**Cockpit** — Red Hat's web-based system administration console, typically on port 9090.

## 9.3 Application stack

---

**SPA** (Single Page Application) — a web application where one HTML page loads and JavaScript handles all subsequent navigation. The employee-facing React app is an SPA.

**Fastify** — A fast, low-overhead web framework for Node.js. Used here in place of Express for better performance and built-in schema validation.

**REST API** — the HTTP-based interface exposed by the backend. Verbs: GET, POST, PATCH, DELETE.

**Session** — a server-side record that identifies a logged-in user. Identified by a cookie the browser sends with every request. Stored in the PostgreSQL Session table.

**Session secret** — a random string used to cryptographically sign session cookies. Stored in `/srv/intranet/.env` as `SESSION_SECRET`.

**bcrypt** — a password hashing algorithm. Slow on purpose, to resist brute-force attacks.

**Plugin** (Fastify) — a piece of functionality registered onto the Fastify instance (auth, session handling, file uploads, rate-limiting). Replaces “middleware” terminology used by Express.

**ORM** — an Object-Relational Mapper. **This project uses Prisma** as its ORM; the application reads and writes the database through Prisma’s generated type-safe client rather than handwritten SQL.

**Prisma** — A modern Node.js ORM. The schema is defined in `prisma/schema.prisma`, migrations live in `prisma/migrations/`, and the generated client (`@prisma/client`) is what the Fastify routes call.

**Migration** (Prisma) — A versioned SQL file that brings the database schema from one state to the next. Applied automatically on backend startup via `prisma migrate deploy`.

## 9.4 Database

---

**PostgreSQL** — A mature, open-source relational database. Runs as a multi-process server on the RHEL host (not in a container in this deployment). Listens on TCP port 5432.

**psql** — PostgreSQL’s command-line client. DBAs and operators use it to run queries.

**pg\_dump / pg\_restore** — PostgreSQL’s logical backup and restore tools. Used for nightly database backups ( `-Fc` produces a compressed custom-format dump that `pg_restore` can apply selectively).

**WAL** (Write-Ahead Log) — PostgreSQL’s transaction log. Every change is written to the WAL before being applied to data files, allowing crash recovery and replication.

**Role / User** (PostgreSQL) — A database principal. The intranet uses: - `intranet_app` — application backend (read/write) - `intranet_ro` — read-only for reporting - `dba_group` — DBA role with full privileges, granted to individual DBA accounts

**pg\_hba.conf** — PostgreSQL’s host-based authentication file. Decides which network ranges and which users can connect, and with what authentication method ( `scram-sha-256` in production).

## 9.5 Containers and Docker

---

**Container** — a sandboxed process running on the host kernel with its own filesystem and network namespace. Not a VM.

**Image** — a snapshot of a filesystem used to start containers. Built from a `Dockerfile`. We use stock public images, no custom Dockerfile.

**Docker Compose** — a tool that defines and runs multi-container apps from a single `docker-compose.yml` file.

**Bind mount** — exposing a host directory inside a container. The host filesystem is canonical; the container sees a window into it.

**Volume** (named) — a Docker-managed storage area. We do not use named volumes; only bind mounts.

**Restart policy** — Docker's instruction for what to do when a container stops. We use `unless-stopped` (auto-restart unless explicitly stopped).

**extra\_hosts** — A Compose directive that injects entries into a container's `/etc/hosts`. We use it to give the backend container a reachable name for the host (`host.docker.internal`).

## 9.6 Security and TLS

---

**TLS** (Transport Layer Security) — the protocol behind HTTPS. Encrypts traffic and verifies server identity.

**Certificate** — A cryptographic document binding a hostname to a public key. Ours is issued by the Huntsville Hospital internal Certificate Authority and trusted by domain-joined workstations.

**Private key** — the secret half of a TLS certificate. Lives at `/etc/nginx/tls/intranet-hci.key`, mode 0600, owned by root.

**HSTS** (HTTP Strict Transport Security) — A response header that tells browsers to use HTTPS only for a given host.

**CSRF** (Cross-Site Request Forgery) — An attack class where a malicious site tricks an authenticated user's browser into making requests to the application. Mitigated by the `SameSite=Lax` cookie attribute used for session cookies.

## 9.7 Virtualization

---

**vSphere** — VMware's enterprise virtualization platform. Huntsville Hospital's production hypervisor environment.

**ESXi** — The hypervisor itself, the bare-metal OS that runs VMs.

**vCenter** — The management interface for one or more ESXi hosts. Where operators go to manage the VM (snapshots, console, power).

**Snapshot** (vSphere) — A point-in-time copy of a VM's disk and memory state. Reverting to a snapshot rolls the VM back to that point.

**OVA / OVF** — Portable VM image formats. Used to migrate VMs between vSphere clusters or to other hypervisors.

## 9.8 Storage

---

**LVM** (Logical Volume Manager) — A Linux abstraction over physical disks. Our root filesystem is on the LVM logical volume `rhel-root`.

**xfs** — The default filesystem on RHEL 10.

**inode** — A filesystem object that holds metadata. Filesystems can run out of inodes even with disk space free (rare).

## 9.9 Frontend

---

**React** — A JavaScript library for building user interfaces. The employee-facing site is a React 19 application.

**React Router** — Routing for single-page apps. Maps URL paths to React components without a full page reload.

**Tailwind** — A utility-class CSS framework. Used in the React frontend.

**Vite** — A modern frontend build tool. We use it to build the React app for production.

**JSX** — JavaScript with embedded XML-like syntax. React component files use this.

## 9.10 Miscellaneous

---

**dotfile** — A file or directory whose name starts with `.`. Hidden by default in `ls`.

**SIGTERM / SIGKILL** — Process signals. SIGTERM asks a process to clean up and exit; SIGKILL terminates immediately. `docker compose stop` sends SIGTERM, then SIGKILL after 10 seconds if the container has not exited.

## 9.11 Common abbreviations used in this document

---

- `cwd` — current working directory
- `pid` — process ID
- `uid / gid` — user / group ID
- `env` — environment (variables)
- `pkg` — package
- `db` — database

- `fs` — filesystem
- `ro / rw` — read-only / read-write
- `FQDN` — fully qualified domain name
- `DBA` — database administrator